AI Full-Stack Developer Guide (LLMs, RAG, Agents, and Production)

This guide is built for interviews where you are expected to design, build, evaluate, and deploy AI applications end-to-end.

1) AI Full-Stack Mental Model

A production AI app is usually:

- Frontend/chat UI.
- API/orchestrator service.
- Model provider(s).
- Retrieval and knowledge layer.
- Tooling/actions layer.
- Observability/evaluation and safety controls.

Interview expectation:

- Explain not only prompt logic but also reliability, latency, and governance.

2) LLM Fundamentals You Must Explain

2.1 Core Concepts

- Tokens and context window.
- Sampling: temperature, top-p, max tokens.
- System/user/tool messages.
- Structured outputs and function/tool calling.

2.2 Why Hallucinations Happen

- Probabilistic next-token generation.
- Missing or stale context.
- Ambiguous prompts.

Mitigations:

- Better retrieval grounding.
- Constrained output schemas.
- Verification/evaluation loops.

3) Model Integration Patterns

3.1 Provider-Agnostic Wrapper Pattern

```
class LLMClient:
    def generate(self, prompt: str) -> str:
        raise NotImplementedError

class OpenAIClient(LLMClient):
    def __init__(self, client, model: str):
        self.client = client
```

```
        self.model = model

    def generate(self, prompt: str) -> str:
        resp = self.client.responses.create(
            model=self.model,
            input=prompt,
        )
        return resp.output_text
```

Interview explanation:

- Hide provider-specific APIs behind adapter interface to reduce lock-in.

3.2 Structured Output with Schema Validation

```
from pydantic import BaseModel

class TicketSummary(BaseModel):
    priority: str
    sentiment: str
    action_items: list[str]
```

Pattern:

- Ask model for JSON.
- Validate response with Pydantic.
- Retry/repair when invalid.

Interview explanation:

- Structured output reduces brittle regex parsing and improves correctness.

4) RAG End-to-End (Interview Core)

4.1 RAG Pipeline Stages

1. Ingestion.
2. Chunking.
3. Embedding.
4. Indexing.
5. Retrieval.
6. Reranking.
7. Augmented prompt.
8. Generation with citations.

4.2 Ingestion and Chunking Example

```
def chunk_text(text: str, size: int = 600, overlap: int = 80):
    out = []
    i = 0
    while i < len(text):
        out.append(text[i:i+size])
```

```
        i += max(1, size - overlap)
    return out
```

Interview explanation:

- Overlap preserves context continuity across chunk boundaries.

4.3 Retrieval Skeleton

```
def retrieve(query: str, top_k: int = 5):
    q_vec = embed(query)
    hits = vector_db.search(vector=q_vec, top_k=top_k)
    return hits
```

4.4 Prompt Augmentation Template

```
QUESTION:
{question}

CONTEXT:
{retrieved_chunks}

Instructions:
- Answer only from CONTEXT.
- If answer is not present, say "I don't know".
- Return citations as source IDs.
```

4.5 Hybrid Retrieval Pattern

- Dense retrieval for semantic similarity.
- Sparse/keyword retrieval for exact terms (IDs, acronyms, error codes).
- Merge + dedupe + rerank.

Interview explanation:

- Hybrid retrieval often improves enterprise search quality.

5) Agentic AI Design

5.1 Agent Components

- Planner (decides next step).
- Tool router (which tool to call).
- Memory/state manager.
- Safety/approval gate.
- Executor and result synthesizer.

5.2 Tool Calling Example (Pseudo)

```
def agent_step(user_query):
    plan = llm_plan(user_query)
```

```
    if plan.tool == "search_docs":
        docs = search_docs(plan.args)
        return llm_answer(user_query, docs)
    if plan.tool == "create_ticket":
        if requires_approval(plan):
            return "Approval required"
        return create_ticket(plan.args)
    return llm_answer(user_query, [])
```

Interview explanation:

- Agents should be explicit state machines, not hidden infinite loops.

5.3 Multi-Agent Workflow Pattern

- Research agent gathers evidence.
- Critic agent validates factual consistency.
- Writer agent produces final response.

Interview explanation:

- Use only when decomposition gives measurable quality gain; avoid complexity without evidence.

6) Memory in AI Apps

6.1 Short-Term vs Long-Term

- Short-term conversation state.
- Long-term user/profile memory with explicit retention policies.

6.2 Memory Safety

- Do not store secrets by default.
- Use redaction and PII controls.
- Allow user-visible memory inspection/deletion.

7) Evaluation and Quality Assurance

7.1 Evaluation Types

- Retrieval quality: recall@k, precision@k.
- Generation quality: factuality, relevance, completeness.
- Task success rate for agent workflows.
- Safety and policy compliance.

7.2 Golden Dataset Pattern

- Build curated question-answer set with expected citations.
- Run before and after changes.
- Track regressions per release.

7.3 LLM-as-Judge (with Caution)

- Useful for scalable scoring.
- Must calibrate with human-reviewed subsets.

8) Guardrails and Security

8.1 Prompt Injection and Tool Abuse

Mitigations:

- strict tool schemas.
- allowlist tool actions.
- content and policy checks before tool execution.
- approval workflows for destructive actions.

8.2 Data Security

- Encrypt data at rest and in transit.
- Enforce per-tenant retrieval boundaries.
- Log access with auditability.

8.3 Output Safety

- Moderate output for policy violations.
- Risk-tier actions before external side-effects.

9) AI Application Latency and Cost Engineering

9.1 Latency Breakdown

- Retrieval latency.
- Model inference latency.
- Tool call latency.
- Post-processing latency.

9.2 Optimization Levers

- Cache embeddings and retrieval results.
- Use smaller model for routing/classification.
- Use larger model only for hard reasoning paths.
- Stream partial outputs.

9.3 Cost Controls

- Token budgets.
- Context compression/reranking.
- Prompt templates with minimal verbosity.
- Request-level and user-level quotas.

10) AI Deployment and MLOps/LLMOps

10.1 Deployment Shapes

- API-only hosted model provider.
- Hybrid (managed model + self-hosted retrieval).
- Self-hosted inference for strict compliance cases.

10.2 CI/CD for AI Apps

- Unit tests for orchestration code.
- Eval suite gate in CI.
- Canary deployments with shadow traffic.
- Rollback on quality or latency regression.

10.3 Observability Dashboard Essentials

- request volume and latency percentiles.
- token input/output per endpoint.
- retrieval hit quality metrics.
- tool-call failure rates.
- hallucination or unsupported-answer rates.

11) End-to-End RAG API Example (FastAPI + Pseudocode)

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/ask")
def ask(q: str):
    hits = retrieve(q, top_k=6)
    context = "\n\n".join([h.text for h in hits])
    prompt = f"QUESTION:\n{q}\n\nCONTEXT:\n{context}\n\nAnswer with citations."
    answer = llm.generate(prompt)
    return {
        "answer": answer,
        "sources": [h.source_id for h in hits],
    }
```

Interview explanation:

- Keep retrieval evidence in output for traceability and trust.

12) High-Frequency AI Interview Questions

1. Why RAG instead of fine-tuning?
- Faster updates, source control, lower retrain overhead for many use cases.
2. When is fine-tuning better?
- Style/task behavior adaptation with stable data and repeatable patterns.
3. How do you evaluate a RAG system?
- Retrieval metrics + answer metrics + human review + business KPIs.
4. How do you prevent prompt injection from retrieved docs?
- Treat retrieved text as untrusted input and isolate from system instructions.
5. How do you handle stale knowledge?
- Scheduled indexing, change streams, and freshness metadata filters.
6. What is agentic RAG?
- Agent controls retrieval and tool choices iteratively to improve final response.
7. How do you ensure deterministic downstream integrations?
- Structured outputs validated by schema + retries + fallback path.
8. How do you reduce cost without killing quality?

- Better retrieval precision, model routing tiers, prompt compaction, caching.

9. How do you deploy safely?

- Offline eval gate, canary, telemetry, rollback automation.

10. What does senior ownership look like in AI systems?

- Not only model calls, but reliability, safety, compliance, and business outcomes.

13) Story Prompts for AI Full-Stack Interviews

- Built production RAG assistant with measurable reduction in support ticket resolution time.
- Improved answer faithfulness by introducing hybrid retrieval + reranking + citations.
- Reduced AI cost by model-routing strategy and context pruning.
- Deployed agent workflow with approval gates and audit logs for safe tool execution.
- Ran post-incident analysis after hallucination event and shipped preventive controls.

14) 90-Minute AI Revision Plan

1. Review sections 2, 4, 5, 7, 8 first.
2. Rehearse one complete architecture answer: ingestion -> retrieval -> generation -> eval.
3. Rehearse one safety answer and one cost-optimization answer.
4. Rehearse two production stories with metrics and rollback details.

15) L4 AI Production Deep Drill

15.1 Evaluation Strategy That Survives Prod

- Offline eval set with versioned golden answers.
- Online metrics: acceptance rate, fallback rate, escalation rate, latency, cost/query.
- Safety evals: policy violation rate, jailbreak success rate, PII leakage checks.

15.2 RAG Retrieval Quality Engineering

- Chunking tuned by document type, not one global heuristic.
- Hybrid retrieval (dense + sparse) plus reranker improves precision.
- Enforce source citation and abstain on low-confidence contexts.

15.3 Agent Reliability Controls

- Tool allowlist and argument schema validation.
- Per-tool timeout, retry budget, and idempotency token.
- Human approval for destructive or high-risk actions.
- Full audit logs for tool calls and model decisions.

15.4 Cost Governance at Scale

- Model routing tiers by task complexity.
- Semantic caching and response reuse for repeated prompts.
- Context compaction and retrieval precision to reduce token waste.

15.5 Senior Grilling Questions

1. What fails first in your AI stack during traffic spike?
2. How do you quantify hallucination reduction after a change?
3. How do you rollback model/prompt/retriever independently?
4. How do you enforce tenant isolation for RAG corpora?